

Database Systems

05

Data Query Language (DQL)



Kak Soky, PhD



soky.kak@cadt.edu.kh



085683680

The SELECT query specifies the columns to be retrieved as a column list.

Syntax:

```
SELECT columnlist  
FROM table  
WHERE condition  
GROUP BY column  
HAVING condition  
ORDER BY column  
LIMIT n
```

id	name	gender	department	business_unit	salary	location
1	Borey	Male	Marketing	Research & Development	933	Preah Vihear
2	Chea	Female	Sales	Manufacturing	832	Preah Sihanouk
3	Heng	Male	IT	Speciality Products	731	Ratanakiri
4	Chankrisna	Female	Finance	Corporate	759	Prey Veng
5	Kiri	Male	Sales	Speciality Products	864	Takeo
6	Mliss	Female	Finance	Speciality Products	250	Pursat
7	Makara	Male	Finance	Speciality Products	1036	Oddar Meanchey
8	Leakhena	Female	Accounting	Corporate	761	Oddar Meanchey
9	Ponnleu	Female	Engineering	Speciality Products	493	Takeo
10	Narin	Male	IT	Manufacturing	731	Prey Veng

```
select id,name,  
gender, salary  
from employee  
where salary<700;
```

id	name	gender	salary
6	Mliss	Female	250
9	Ponnleu	female	493

Regular Expressions help search data matching complex criteria.

Basic syntax

```
SELECT statements...  
WHERE fieldname REGEXP 'pattern';
```

Pattern	Match Description
*	Zero or more instances of string preceding it
+	One or more instances of strings preceding it
.	Any single character
?	Match zero or one instances of the strings preceding it.
^	caret (^) matches Beginning of string
\$	End of string
[abc]	Any character listed between the square brackets
[^abc]	Any character not listed between the square brackets
[A-Z]	match any upper-case letter.
{m,n}	m through n instances of preceding element

Example

```
SELECT id, name, gender  
FROM employee  
WHERE name REGEXP '[nr].{2}';
```

id	name	gender
1	Borey	Male
4	Chankrisna	Female
9	Ponnleu	Female
10	Narin	Male

```
SELECT department, count(*)  
as total FROM employee  
GROUP BY department;
```

department	total
Marketing	1
Sales	2
IT	2
Finance	3
Accounting	1
Engineering	1
Marketing	1

```
SELECT department, count(*) as total  
FROM employee  
GROUP BY department  
HAVING count(*) > 1;
```

department	total
Sales	2
IT	2
Finance	3

- The **GROUP BY** clause is a SQL command that is used to group rows that have the same values.
- The **HAVING** clause is used in the SELECT statement to specify filter conditions for a group of rows or aggregates.

MySQL databases generally store a large amount of data. To analyze this data effectively, DBAs and Analysts need to pull records from two or more tables based on specific conditions.

- JOIN combines records from two tables.
- A query can contain zero, one, or multiple JOIN operations.

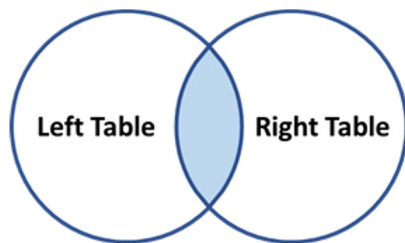
supplier

id	name
1	Aliexpress
2	Alibaba
3	Toobao

invoice

id	sid	date
1	2	2023-01-02
2	2	2023-01-03
3	1	2023-01-04

- An INNER JOIN matches column values in two tables.
- INNER JOIN is the same as JOIN; the keyword INNER is optional.



supplier

INNER JOIN

invoice

id	name
1	Aliexpress
2	Alibaba
3	Toobao

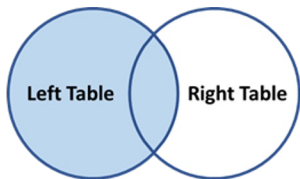
id	sid	date
1	2	2023-01-02
2	2	2023-01-03
3	1	2023-01-04



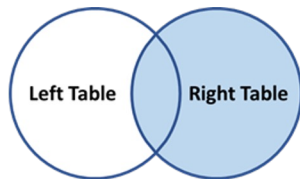
```
SELECT s.id, name, date  
FROM supplier s  
JOIN invoice on s.id = sid;
```

id	sid	date
1	Aliexpress	2023-01-04
2	Alibaba	2023-01-02
2	Alibaba	2023-01-03

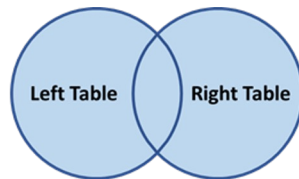
- Rows without a match will have NULL column values.
- Possible OUTER JOIN styles are LEFT, RIGHT, or FULL.
- The OUTER keyword is optional



LEFT OUTER JOIN



RIGHT OUTER JOIN



FULL OUTER JOIN

id	name
1	Aliexpress
2	Alibaba
3	Toobao

id	sid	date
1	2	2023-01-02
2	2	2023-01-03
3	1	2023-01-04

```
SELECT s.id, name, date
from supplier s
LEFT JOIN invoice on s.id= sid;
```

id	sid	date
1	Aliexpress	2023-01-04
2	Alibaba	2023-01-02
2	Alibaba	2023-01-03
3	Toobao	NULL

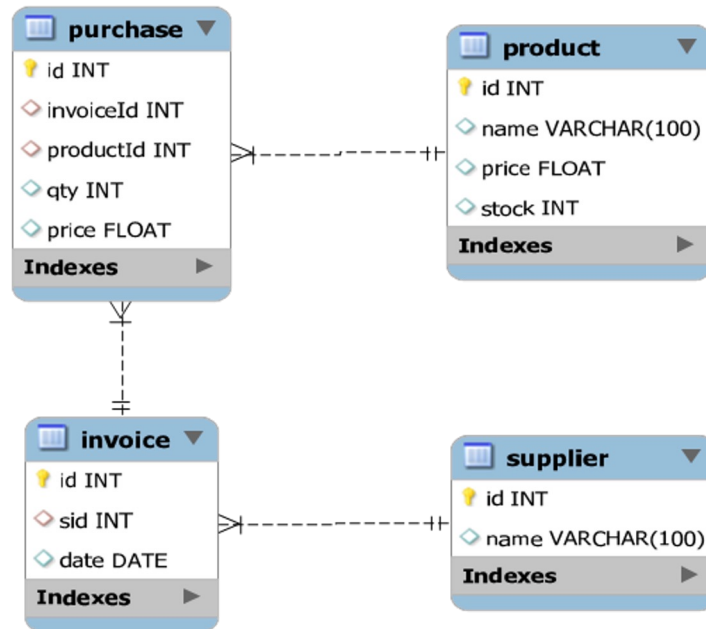
SELECT s.name, i.id, purchase.id, product.name, qty,
purchase.price

FROM supplier s

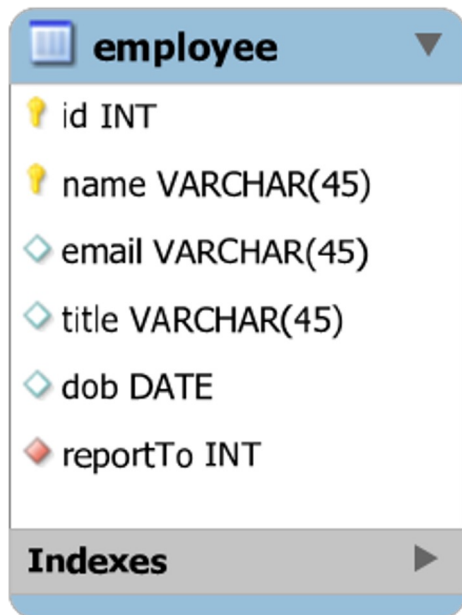
JOIN invoice i **on** s.id = i.sid

JOIN purchase **on** purchase.invoiceId = i.id

JOIN product **on** product.id = purchase.productId;



A self join is a regular join, but the table is joined with itself.



```
SELECT e.name, m.name
```

```
FROM employee e
```

```
JOIN employee m ON e.id= m.reportTo;
```

A MySQL subquery is a query nested within another query such as SELECT, INSERT, UPDATE or DELETE.

```
SELECT column_list (s) FROM table_name  
WHERE column_name OPERATOR  
      (SELECT column_list (s) FROM table_name [WHERE])
```

For example, the following query returns the oldest employee.

```
SELECT name, dob  
FROM employee  
WHERE dob = (SELECT MIN(dob) FROM employee);
```

Aggregate functions let you perform a calculation on multiple data and return a single value. MySQL supports all the five (5) ISO standard aggregate functions.

COUNT	Returns the number of rows with non-null values for a given column
MIN	Returns the minimum attribute value found in a given column
MAX	Returns the maximum attribute value found in a given column
SUM	Returns the sum of all values for a given column
AVG	Specifies the tables from which data should be retrieved

String Functions
CONCAT
LEFT
LENGTH
LOWER
REPLACE
REVERSE
RIGHT
SUBSTR
UPPER

Numeric Functions
CEIL
FLOOR
GREATEST
LEAST
POW
ROUND
SQRT
SUM
TRUNCATE

Date Functions
ADDDATE
ADDTIME
CURRENT_DATE
CURRENT_TIME
DATE_FORMAT
DATEDIFF
DAY
DAYNAME
NOW

Advanced Functions
CASE
IF
IFNULL
ISNULL
USER

CASE() function in MySQL is used to find a value by passing over conditions whenever any condition satisfies the given statement otherwise it returns the statement in an else part.

Syntax:

```
CASE case_value  
  
    WHEN expression THEN statement_list  
  
    [WHEN expression THEN statement_list] ...  
  
    [ELSE statement_list]  
  
END
```

Example:

```
SELECT score, grade,  
CASE  
    WHEN score >= 90 THEN 'A'  
    WHEN score >= 80 AND score < 89 THEN 'B'  
    WHEN score >= 70 AND score < 79 THEN 'C'  
    WHEN score >= 60 AND score < 69 THEN 'D'  
    WHEN score >= 40 AND score < 59 THEN 'E'  
    ELSE 'F'  
END as class  
FROM score;
```

A common table expression (CTE) is a named temporary result set that exists within the scope of a single statement and that can be referred to later within that statement, possibly multiple times.

Example:

```
WITH  
  i AS (select date, sid from invoice),  
  s AS (select * from supplier)  
SELECT i.date, s.name from i JOIN s  
ON i.sid = s.id;
```